CSED233 · 데이터구조 · 포스텍

Ch1. Introduction — 자료구조란?

1. Why Data Structure(자료구조)?

핵심 이론 & 배경

- 핵심 관점: 프로그램은 데이터를 저장·조직·접근하는 자료구조(Data Structure)와 데이터를 처리하는 알고리즘(Algorithm)을 함께 고려하여 설계한다. 이를 기억하기 위한 개념적 표현이 Program = Data Structure + Algorithm이다. 이는 수학적 항등식이 아니라 자료구조와 알고리즘의 상호보완적 역할을 강조하는 요약이다.
- **Data Structure**: 정보(data)를 어떻게 **조직(organize)** 하는가
- **Algorithm(알고리즘)**: 프로그램이 task를 어떻게 **수행**하는가
- Why 배워야 하는가?: 기존 라이브러리나 AI 코딩 도구가 문제의 일부 또는 전체를 해결할 수 있지만, 입력 규모, 연산 요구사항, 시간·메모리 제약 등이 달라지면 기존 해법이 적합하지 않을 수 있다. 따라서 자료구조의 동작과 성능 특성을 이해하고, 문제에 맞게 선택·활용·수정·구현할 수 있어야 한다.

직관 & 예시

- AI 모델과 데이터 표현의 관계: AI 모델마다 특정 데이터 표현과 구조적 특성을 효과적으로 처리하도록 설계된 경우가 있다. 아래는 대표적인 대응 사례이며, 하나의 데이터 형태에 하나의 모델만 사용할 수 있다는 의미는 아니다.

Data structure	Related AI model
Grid(격자)	CNN
Sequence(시퀀스)	Transformer/RNN/LSTM/GRU
Set(집합)	Deep Sets
Tree(트리)	TreeLSTM
Point Cloud(점 구름·점 집합)	PointNet

Graph(그래프)	GNN

- → 모델·알고리즘 선택 원칙: 먼저 데이터의 구조와 표현 방식을 파악하고, 수행할 작업, 입력 규모, 정확도 요구사항, 시간·메모리 제약을 함께 고려하여 적합한 자료구조와 알고리즘을 선택한다. 데이터 형태와 AI 모델의 대응 관계는 후보를 좁히는 데 유용하지만, 실제 적합성이나 성능을 보장하지는 않는다.

2. Course Roadmap (학습 지도)

전체 목차

1. **C++** — 실습 기반
2. Algorithm Analysis — 입력 크기 n에 따른 시간·공간 사용량의 증가율 분석. Big-O 표기법은 충분히 큰 n에서 함수의 점근적 상한을 나타낸다. 분석할 때는 비용 측정 기준과 최악·평균·최선 중 어떤 경우인지 구분하며, 점근적 복잡도 비교가 실제 하드웨어에서의 실행시간이나 모든 입력 크기에서의 성능 우위를 보장하지는 않는다.
3. **List(리스트) / Stack(스택) / Queue(큐)** — 선형 자료구조
4. **Tree** — 계층 구조
5. **Priority Queue(우선순위 큐) / Heap(힙)** — 우선순위 기반
6. **Sorting(정렬)** — 정렬
7. **Special Trees(트리)** — BST, AVL, …
8. Graph — 정점(vertex)과 간선(edge)으로 개체 사이의 관계를 표현하는 일반적인 자료구조. 계층 관계와 비계층 관계를 모두 나타낼 수 있으며, 모든 정점 쌍이 연결되어 있을 필요는 없다.
9. **Shortest Path(최단 경로) / MST**
10. **String** — Trie(트라이), suffix tree, pattern matching

각 주제의 motivational question

- C++: 자료구조를 클래스·포인터·동적 메모리 등으로 어떻게 구현할 것인가?
- Algorithm Analysis: 같은 문제를 해결하는 알고리즘들을 입력 크기에 따른 시간·공간 사용량으로 어떻게 비교할 것인가?
- List / Stack / Queue: 위치에 따라 원소를 관리하는 List, 나중에 넣은 원소를 먼저 꺼내는 Stack(LIFO), 먼저 넣은 원소를 먼저 꺼내는 Queue(FIFO)는 각각 언제 적합한가?
- Tree: 조직도나 디렉터리처럼 부모·자식 관계가 있는 데이터를 어떻게 표현하고 탐색할 것인가?

- Priority Queue / Heap: 우선순위가 높은 원소를 효율적으로 꺼내려면 어떻게 해야 하는가? 우선순위 큐는 연산을 정의하는 추상 자료형이고, 힙은 이를 구현하는 대표적인 방법이다.
- Sorting: 원소들을 지정한 비교 기준에 따라 오름차순 또는 내림차순으로 어떻게 정렬할 것인가?
- Special Trees: BST에서 효율적으로 검색·삽입·삭제하려면 어떤 구조가 필요한가? AVL 트리는 높이의 균형을 어떻게 유지하는가?
- Graph: 임의의 개체 사이에 존재하는 연결 관계를 어떻게 표현하고 탐색할 것인가?
- Shortest Path: 두 정점 사이에서 경로의 가중치 합을 최소화하려면 어떻게 할 것인가?
- MST: 연결된 무방향 가중 그래프의 모든 정점을 사이클 없이 최소 총 가중치로 연결하려면 어떻게 할 것인가?
- String: 접두사 검색, 부분 문자열 탐색, 패턴 매칭을 어떻게 효율적으로 수행할 것인가?